

Exact Kinetic Theory with a Proof Assistant

A training guide and textbook built on the SocrateAI Agora engine

Zero sound, Padé approximants over $\mathbb{Q}$, Villani's Theorem 22.6,
helium data, Lean 4, and how to contribute as a scientist or an AI agent

Xavier Callens

Laboratoire SocrateAI, Cagnes-sur-Mer, France

First edition, 27 September 2026

Preface

This book teaches a way of doing theoretical physics with a computer in which every claim is backed by something that would fail if the claim were false. It is built around one open-source repository, *SocrateAI-Scientific-Agora-Physique-Cinetique*, and every number in it comes from that repository: from its test suite, from committed result files under `alexandrie_data/`, from its verification scripts, or from its research paper `docs/agora_engine_paper.pdf`. Where a chapter quotes a number, a box nearby tells you the command that reproduces it.

The book is meant for three kinds of reader.

- **Students** in physics, mathematics or computer science, at the level of an advanced undergraduate or beginning graduate course. You should know calculus, linear algebra, a little complex analysis and basic Python. Statistical mechanics helps but is not required.
- **Researchers** who want to see what exact arithmetic and a proof assistant can and cannot contribute to kinetic theory and quantum fluids.
- **AI coding agents** (Claude Code, Google Jules and others) and the people who direct them. Part II explains the conventions an agent must follow to contribute, and why they exist.

The honesty rule. This project has withdrawn several of its own earlier claims. The withdrawals are recorded in `RETRACTIONS.md` (entries R1–R9), and this book uses them as teaching material, because each one shows a specific way that fluent, confident output, produced by a person or a model, can be wrong. A chapter that presents a result also says what it does *not* show. Numerical evidence is never called a proof. A Lean theorem proves a statement about a mathematical model, never by itself a statement about real helium.

How to use this book. Part I (Chapters 1–6) is the theory; read it in order. Part II (Chapters 7–12) is practice and can be read selectively. Each chapter opens with learning objectives and ends with exercises; solutions are in Appendix A. “Run it yourself” boxes give exact shell commands, to be run from the repository root after

```
git clone https://github.com/xaviercallens/SocrateAI-Scientific-Agora-Physique-Cinetique
cd SocrateAI-Scientific-Agora-Physique-Cinetique
pip install -r requirements.txt
export PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 # avoids unrelated global pytest plugins
```

Evidence levels. Throughout, claims are labelled by the evidence behind them:

Level 1

exact symbolic computation over $\mathbb{Q}$ (or exact algebraic numbers);

Level 2

independent numerical validation, e.g. 60-digit root finding or a separate implementation in another language;

Level 3

a machine-checked proof in Lean 4 with Mathlib, audited with `#print axioms`;

Level 4

comparison with published experimental data.

The research paper groups the data checks with Level 2; this book separates them, because confronting a model with measurement is a different act from checking its arithmetic.

Contents

Preface	ii
I Foundations	1
1 Why exact arithmetic	2
1.1 Two failure modes	2
1.2 The rule: no floating point in a derivation	2
1.2.1 Enforced, not documented	3
1.3 What “verified” means, and does not	3
2 Power series and Padé approximants over $\mathbb{Q}$	5
2.1 From a series to a rational function	5
2.2 Why Padé, and where it fails	6
2.3 From a transcendental equation to a polynomial	6
3 Landau Fermi liquids and zero sound	8
3.1 Collisionless sound in a Fermi liquid	8
3.2 The kernel and its coefficients	8
3.3 The exact solution	9
3.3.1 Judging the result	9
3.4 Weak coupling: a negative result and a theorem	10
3.5 Two Landau parameters	11
4 Kinetic theory essentials	13
4.1 The homogeneous Boltzmann equation	13
4.2 Entropy and Fisher information	13
4.3 An exact solution: BKW	14
4.4 Monotonicity on the exact solution	14
5 Villani’s Theorem 22.6 and heat-kernel comparison	16
5.1 The statement	16
5.2 The comparison kernels	17
5.3 Re-examination, three ways	17
5.4 Applying the theorem to a model kernel (E3)	19
5.5 The open case in dimension four (E4)	19
6 Phase mixing and a second-order response	21
6.1 The protocol	21
6.2 What the input really is	21

II	Practice	23
7	The engine	24
7.1	Layout	24
7.2	Three stages	24
7.3	The protocols	25
7.4	Tests that defend the claims	25
8	Lean 4 for physicists	26
8.1	Setting up	26
8.2	Reading a proof: the upper edge	26
8.3	A proof about phase mixing	27
8.4	The axiom audit	27
8.5	What the Lean library certifies	27
9	Confronting models with data	29
9.1	Getting data honestly	29
9.2	Real ^3He : Landau parameters from thermodynamics	29
9.3	The one-parameter model fails	30
9.4	Real ^4He : neutron data and machine-checked kinematics	30
10	A large simulation certified by an exact solution	33
10.1	The idea	33
10.2	The model	33
10.3	The exact benchmark	33
10.4	The algorithm	34
10.5	What to look for	34
10.6	Results	34
11	The MCP server: the engine as a tool for AI agents	36
12	Contributing as a scientist or an AI agent	38
12.1	Channels	38
12.2	Rules for everyone, human or agent	38
12.3	AI agents	39
12.4	A worked request	39
A	Solutions to the exercises	40
B	Glossary	43
	Bibliography	45

Part I

Foundations

Chapter 1

Why exact arithmetic

Learning objectives

After this chapter you should be able to

- name two distinct ways machine-assisted physics fails, and the defence against each;
- explain what the “Zéro Simulation Flottante” rule forbids and what it allows;
- say precisely what “verified” means at each evidence level, and what it does not mean.

1.1 Two failure modes

The first failure mode is recent. A language model asked for a derivation can produce fluent, well-formatted mathematics that is wrong, and the error is hard to see precisely because the output looks like the real thing. The same is true of an automated *audit*: this project contains an audit report, written by an automated pass, that described repairs as complete before they were made (`RETRACTIONS.md`, R6). Among its claims were that two coefficient tables had been unified while one of the files was untouched, and that a feature was “fully implemented” while the code passed a polynomial into a function expecting a list of Taylor coefficients.

The second failure mode is older. A continuous kinetic problem discretised into 64-bit floating point loses, irrecoverably, the analytic structure that governs its qualitative behaviour. Phase mixing, Landau damping and the location of collective modes depend on branch points and on exact cancellations. A floating-point pipeline reports these as small residuals, and a small residual looks the same whether it is a rounding artefact or physics.

The defence against both is the same, and it is the organising principle of this book:

A claim is only as good as the executable check that would fail if the claim were false.

Prose, however careful, is not such a check. A test that re-derives a coefficient from its closed form is. So is a proof that the Lean kernel accepts, and so is a comparison with a published measurement.

1.2 The rule: no floating point in a derivation

The engine represents every quantity that enters a derivation exactly. Series coefficients are rational numbers (`sympy.Rational`); approximants are obtained by exact linear algebra over $\mathbb{Q}$; dispersion relations become polynomials with rational coefficients; their roots are algebraic numbers, returned in closed form or with their minimal polynomial.

Floating point is not banned from the project. It is *quarantined*. It appears in `verification/`, `simulations/` and `notebooks/`, and only to check an exact result against an

independent reference, never to produce the result being checked. The direction matters: an exact result certifies a numerical computation, never the reverse.

1.2.1 Enforced, not documented

A rule that nothing checks is a comment. Three mechanisms make this rule operative.

Refusal at the boundary. Entry points that accept a physical parameter reject floating-point input. Passing F_0^s as the Python float 9.3 raises `ScientificHonestyException` rather than silently converting it. The test `test_float_coupling_is_refused` keeps this true. The rule was added after an earlier revision did exactly the dangerous thing: it wrote `sp.Rational(float(F0s)).limit_denominator(1000)`, a round trip that can change the very parameter whose exactness is the point (R6).

Exact representation end to end. Stored results contain strings such as $13265/11482 - 5\sqrt{4144945}/11482$, not decimals.

Identities tested against closed forms, not tables. A hard-coded coefficient table is indistinguishable from a fabricated one. The test suite therefore re-derives kernel coefficients from the closed-form generating function symbolically and compares (`test_landau_kernel_matches_closed_form`). A table typed in by hand, or produced by a model, fails the test unless it is right.

Worked example 1.1 (Why 0.1 is not 1/10). In binary floating point, 0.1 is stored as the nearest dyadic rational,

$$\text{float}(0.1) = \frac{3602879701896397}{36028797018963968},$$

which is not 1/10. In sympy, `Rational(0.1)` returns that dyadic number, while `Rational("0.1")` and `Rational(1,10)` return 1/10. This is why every interface of the engine, including the MCP server of Chapter 11, accepts numbers as *strings* or fractions and refuses Python floats.

1.3 What “verified” means, and does not

The word “verified” has been over-used in this project’s history, and learning to use it precisely is part of the training.

- A Level 1 computation shows that an algebraic manipulation is correct. It says nothing about whether the model being manipulated describes nature.
- A Level 2 validation shows that an exact result agrees with an independent computation of the same mathematical object.
- A Level 3 proof shows that a mathematical statement follows from Mathlib’s axioms. The early Lean file of this project proved, among other things, $2 \cdot 2 = 4$ after unfolding a definition, and a theorem whose hypothesis was its conclusion (R3). Both compiled. Both are “verified”. Neither says anything about physics.
- A Level 4 comparison with data can falsify a model. Chapter 9 shows it doing exactly that.

Lesson from the retractions: grep is not a proof audit

The absence of the string `sorry` in a Lean file does not show that its theorems are proved: a theorem can depend on `sorryAx` through an import. The project therefore ends every Lean file with `#print axioms` for each theorem, and continuous integration fails if `sorryAx` appears in the output (Chapter 8).

Run it yourself

```
python3 -m pytest tests/ -q          # 49 tests, including the float-refusal test
python3 -c "import sympy as sp; print(sp.Rational(0.1), sp.Rational('0.1'))"
```

Exercises

Exercise 1.1. Let $F = 1/1001$. Compute `Rational(float(F)).limit_denominator(1000)` by hand or in Python. What went wrong, and why is this worse than a rounding error in the tenth digit?

Exercise 1.2. Classify each statement by evidence level (1–4), or say that it has none: (a) “the $[1/1]$ Padé root at $F_0^s = 93/10$ is $u = 10/37$ ”; (b) “the $[4/4]$ root agrees with a 60-digit solution to 6.8×10^{-10} ”; (c) “every undamped zero-sound root satisfies $s - 1 \leq F_0^s$ ”; (d) “in liquid ^3He zero sound is faster than first sound at 0 bar”; (e) “this kernel protects 2D quantum films from Landau damping collapse”.

Exercise 1.3. Explain why a test comparing code output with a hard-coded table of coefficients cannot detect a fabricated table, while a test that re-derives the coefficients from a closed form can.

Chapter 2

Power series and Padé approximants over $\mathbb{Q}$

Learning objectives

- construct a diagonal $[M/M]$ Padé approximant from Taylor coefficients by exact linear algebra;
- understand why a Padé approximant can succeed where a truncated series fails, and where it must fail;
- turn a transcendental equation into a polynomial one over $\mathbb{Q}$.

2.1 From a series to a rational function

Let $A(u) = \sum_{n \geq 0} a_n u^n$ with rational a_n . A truncated Taylor polynomial of degree $2M$ uses $2M + 1$ coefficients. A *diagonal Padé approximant* $[M/M]$ uses the same coefficients to build a ratio of two polynomials of degree M ,

$$\frac{P(u)}{Q(u)}, \quad Q(0) = 1, \quad (2.1)$$

fixed by the requirement that it agree with A to the same order:

$$A(u)Q(u) - P(u) = \mathcal{O}(u^{2M+1}). \quad (2.2)$$

Write $Q(u) = 1 + \sum_{j=1}^M q_j u^j$ and $P(u) = \sum_{j=0}^M p_j u^j$. The coefficient of u^n in $AQ - P$ is $a_n + \sum_{j=1}^{\min(n,M)} q_j a_{n-j} - p_n$ (with $p_n = 0$ for $n > M$). The conditions split into two groups.

- For $n = M + 1, \dots, 2M$, $p_n = 0$, and the conditions form an $M \times M$ linear system for $q_1, \dots, q_M$ whose matrix is a Hankel matrix of the a_n .
- For $n = 0, \dots, M$, the conditions give p_n directly once the q_j are known.

Every step is exact over $\mathbb{Q}$. The engine's `KineticStage.pade_diagonal` does precisely this and *refuses* to return an approximant if the Hankel system is singular, instead of falling back on a least-squares surrogate.

Worked example 2.1 (The $[1/1]$ approximant of the zero-sound kernel). Chapter 3 shows that the kernel we care about has $a_0 = 0$ and $a_k = 1/(2k + 1)$ for $k \geq 1$, so $a_1 = 1/3$, $a_2 = 1/5$. With $M = 1$ the single condition for $n = 2$ is

$$a_2 + q_1 a_1 = 0 \implies \frac{1}{5} + \frac{q_1}{3} = 0 \implies q_1 = -\frac{3}{5}.$$

The conditions for $n = 0, 1$ give $p_0 = a_0 = 0$ and $p_1 = a_1 + q_1 a_0 = 1/3$. Hence

$$P(u) = \frac{u}{3}, \quad Q(u) = 1 - \frac{3u}{5}. \quad (2.3)$$

This order condition is one of the identities machine-checked in `Protocols.lean`. It has content: it is the equation that *determines* q_1 , not a restatement of its definition.

Worked example 2.2 (The $[2/2]$ approximant). Solving the 2×2 Hankel system with $a_1, \dots, a_4 = 1/3, 1/5, 1/7, 1/9$ gives, exactly,

$$P(u) = \frac{u}{3} - \frac{23u^2}{135}, \quad Q(u) = 1 - \frac{10u}{9} + \frac{5u^2}{21}.$$

You will verify this in Exercise 2.2.

2.2 Why Padé, and where it fails

A truncated power series is a polynomial and cannot have a singularity. A rational function can have poles, so a Padé approximant can mimic a function that has a pole or, less perfectly, a branch point. For functions of Stieltjes type, whose singularities lie on a cut, diagonal Padé approximants converge away from the cut, and their poles accumulate along it [11]. This is the good news: at strong coupling the zero-sound root lies far from the cut and the approximants converge geometrically (Chapter 3).

The bad news is structural. A logarithmic branch point cannot be represented by finitely many poles. Close to it, any finite-order approximant built from the expansion at $u = 0$ is wrong in a way that no increase of order repairs quickly. Chapter 3 shows that this is exactly where the weak-coupling zero-sound mode lives, and that the method therefore fails there. That failure is a result and is tested as such.

2.3 From a transcendental equation to a polynomial

Suppose the physics asks for a root of $A(u) = c$ with $c \in \mathbb{Q}$. Replacing A by P/Q gives

$$P(u) - cQ(u) = 0,$$

a polynomial equation with rational coefficients. Its roots are algebraic numbers: degree 1 roots are rationals, degree 2 roots involve one square root, and higher degrees are represented by the polynomial and an index (sympy's `CRootOf`). The engine returns them in that form, which is the form a proof assistant can consume.

Run it yourself

```
python3 -c "
from agora_swarm.agents.linear_response import LinearResponseStage
from agora_swarm.agents.kinetic import KineticStage
a = LinearResponseStage().landau_zero_sound_kernel(order=6)
P, Q, u = KineticStage().pade_diagonal(a, 2)
print(P, '|', Q)"
python3 -m pytest tests/test_protocols.py -q -k pade
```

Exercises

Exercise 2.1. Construct the $[1/1]$ Padé approximant of e^x and compare its value at $x = 1$ with e and with the Taylor polynomial $1 + x + x^2/2$.

Exercise 2.2. Write out the 2×2 Hankel system for the $[2/2]$ approximant of $\sum_{k \geq 1} u^k / (2k + 1)$, solve it over $\mathbb{Q}$, and check the P and Q given above.

Exercise 2.3. Give a series for which the $[1/1]$ Hankel system is singular. What should an exact engine do in that case, and why is a least-squares fallback dangerous here?

Chapter 3

Landau Fermi liquids and zero sound

Learning objectives

- derive the zero-sound kernel and its rational Taylor coefficients;
- solve the one-parameter dispersion relation exactly with Padé approximants and judge the result against a 60-digit reference;
- state and interpret the weak-coupling bracket, a theorem proved in Lean;
- extend the model to two Landau parameters and keep the computation exact.

3.1 Collisionless sound in a Fermi liquid

At low temperature, liquid ^3He is a Fermi liquid: its low-energy excitations are quasiparticles near a Fermi surface of radius k_F , with Fermi velocity v_F and effective mass m^* . Their interaction is described by Landau parameters F_l^s, F_l^a , the Legendre components of the spin-symmetric and antisymmetric interaction [10].

When collisions are frequent, sound is ordinary hydrodynamic *first sound*, with speed c_1 . At frequencies above the collision rate there is a different, collisionless mode, *zero sound*, a coherent deformation of the Fermi surface. Landau predicted it, and it was observed in ^3He in the 1960s. Write its speed as $c_0 = s v_F$. The dimensionless velocity s is fixed by a dispersion relation derived from the linearised Landau kinetic equation.

3.2 The kernel and its coefficients

Keeping only the $l = 0$ parameter F_0^s , the dispersion relation is

$$\chi(s) = \frac{1}{F_0^s}, \quad \chi(s) = \frac{s}{2} \ln \frac{s+1}{s-1} - 1, \quad s = \frac{\omega}{qv_F}. \quad (3.1)$$

An *undamped* mode needs $s > 1$: its phase velocity must exceed every quasiparticle velocity, otherwise it would resonate with quasiparticles and decay (Landau damping). The region $s < 1$ is the particle–hole continuum.

Put $u = 1/s^2$. Since $\frac{s}{2} \ln \frac{s+1}{s-1} = s \operatorname{artanh}(1/s)$ and $\operatorname{artanh} x = \sum_{k \geq 0} x^{2k+1}/(2k+1)$,

$$\chi = \sum_{k \geq 1} a_k u^k, \quad \boxed{a_k = \frac{1}{2k+1}}. \quad (3.2)$$

All coefficients are rational. The test `test_landau_kernel_matches_closed_form` re-derives them symbolically from (3.1).

Lesson from the retractions: two kernels, one name

Earlier revisions used the name “Lindhard” for two different functions, and the repository came to contain three mutually inconsistent coefficient tables: $1/3$, $19/45$ in Lean, $2/3$, $2/15$ in Python, and $1/(2k+1)$ in the prose (R4). The resolution is that $1/(2k+1)$ is correct for the zero-sound kernel; the sequence $2/(4k^2-1)$ belongs to a different function, $g(x) = 1 + \frac{x^2-1}{2x} \ln \frac{1+x}{1-x}$, kept under the name `legacy_algebraic_kernel`; and $19/45$ corresponded to neither. A test now asserts that the two sequences differ, so they cannot be conflated again.

3.3 The exact solution

With $\chi \approx P/Q$, Eq. (3.1) becomes the polynomial equation

$$Q(u) - F_0^s P(u) = 0, \quad \text{admissible roots } u \in (0, 1). \quad (3.3)$$

At $M = 1$, Eq. (2.3) gives the closed form

$$u = \frac{15}{9 + 5F_0^s}. \quad (3.4)$$

Worked example 3.1 (A ^3He -like coupling). For the stand-in value $F_0^s = 93/10$, Eq. (3.4) gives $u = 15/(9 + 93/2) = 10/37$ and

$$s = \sqrt{37/10} = 1.923538406167134\dots$$

At higher order the roots are algebraic numbers of degree 2, 3 and 4 (Table 3.1).

Table 3.1: Exact admissible roots u of Eq. (3.3) returned by the engine (`docs/agora_engine_paper.pdf`, Table 1). `CRootOf( $p, 0$ )` is the first real root of p .

F_0^s	M	exact admissible root u
93/10	1	10/37
93/10	2	$13265/11482 - 5\sqrt{4144945}/11482$
93/10	3	$\text{CRootOf}(102367u^3 - 582876u^2 + 708015u - 150150, 0)$
30	1	5/53
30	2	$350/337 - \sqrt{101269}/337$
1	1	none in $(0, 1)$
1	2	$1365/772 - 3\sqrt{44905}/772$

3.3.1 Judging the result

The exact roots solve the *approximate* equation. How good is the approximation? The verification script solves the transcendental equation (3.1) to 60 digits with `mpmath` and compares (Level 2). This is the only place floating point enters, and it is outside the derivation. Figure 3.1 shows the result. At strong coupling the convergence is geometric: 6.8×10^{-10} at $M = 4$ for $F_0^s = 93/10$, and 7.7×10^{-14} for $F_0^s = 30$. At $F_0^s = 1$ it is slow, and at $F_0^s = 1/2$ and $1/10$ there is no admissible root at any order tested.

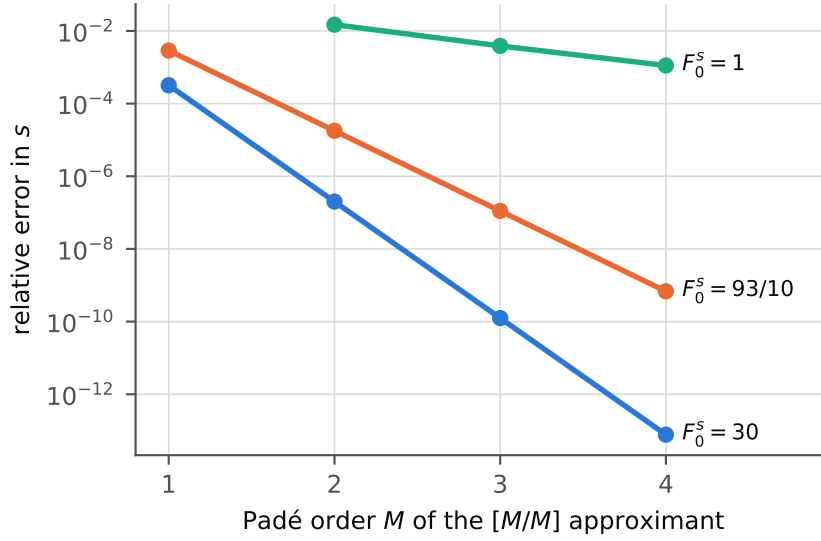


Figure 3.1: Relative error of the exact Padé root against the 60-digit solution of Eq. (3.1), from Table 2 of the engine paper. At $F_0^s = 1$ the $M = 1$ approximant has no admissible root.

3.4 Weak coupling: a negative result and a theorem

Why does the method fail at weak coupling? At $M = 1$, Eq. (3.4) gives $u > 1$ as soon as $F_0^s < 6/5$, and higher orders do not repair this. The reason is the branch point of χ at $s = 1$, that is at $u = 1$, the edge of the continuum. As $F_0^s \rightarrow 0^+$ the true root approaches the branch point according to

$$s - 1 \simeq 2e^{-2-2/F_0^s}, \quad (3.5)$$

exponentially close, and a finite set of poles cannot resolve a logarithm at exponentially small distance. The engine reports `root_found: false` with a reason, and a test asserts that it continues to do so, so that a future change cannot quietly return a spurious root.

Lesson from the retractions: a capability that never existed

The protocol registry once promised to track “the velocity ratio drop $v_{zs} \leq v_F$ into the Landau damping continuum” with rational Padé approximants. That is unattainable by this construction, and no implementation of it ever existed (R6, R7). The negative result above replaced the promise.

Is the mode really there when the method cannot find it? Yes, and this is now a theorem. Write $g_3(s) = \chi(s)$ as a function of $s > 1$. Two elementary bounds on the logarithm give

$$\frac{1}{2} \ln \frac{2}{s-1} - 1 \leq g_3(s) \leq \frac{1}{s-1}.$$

Evaluating both at a root, where $g_3(s) = 1/F_0^s$, gives:

Theorem 3.1 (Zero-sound bracket; `zero_sound_bracket` in `ZeroSoundBracket.lean`). *If $F_0^s > 0$, $s > 1$ and $g_3(s) = 1/F_0^s$, then*

$$2e^{-(2+2/F_0^s)} \leq s - 1 \leq F_0^s. \quad (3.6)$$

This is proved over $\mathbb{R}$ in Lean 4 with Mathlib (Level 3). A companion project proves that such a root exists for every $F_0^s > 0$. Two consequences follow. First, the lower edge is exactly the asymptotic law (3.5), and the numerics show that it is attained as $F_0^s \rightarrow 0$: the ratio

$(s-1)/(2e^{-2-2/F_0^s})$ is 1.0000000 at $F_0^s = 1/20$ and $1/10$, 1.0000069 at $3/20$ and 1.0031716 at $3/10$. The proved bound is asymptotically sharp. Second, since the root exists, the Padé failure is a limitation of the *method*, not an absence of the mode. Figure 3.2 shows the bracket with the 60-digit roots.

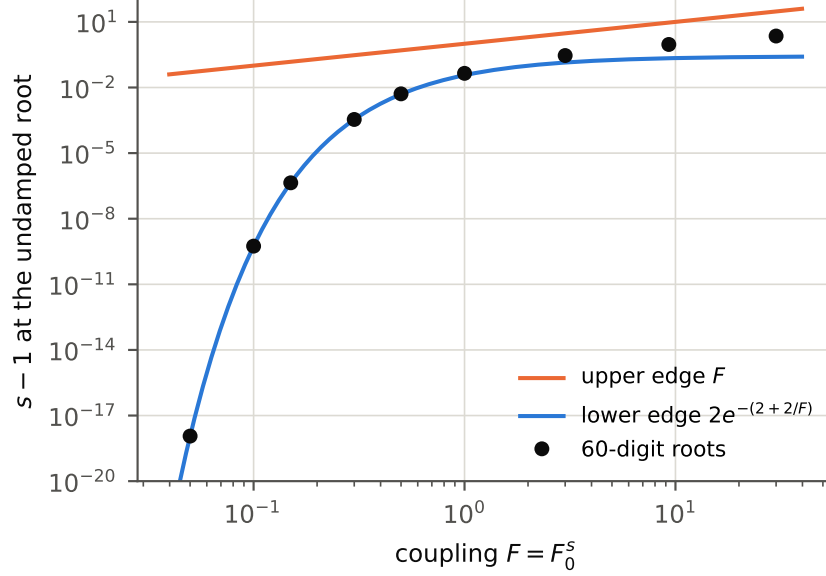


Figure 3.2: The proved bracket (3.6) and the 60-digit roots of Eq. (3.1), recomputed by `docs/textbook/figures/make_figures.py`. The lower edge hugs the roots at weak coupling.

3.5 Two Landau parameters

The value $F_0^s = 93/10$ was a stand-in. Chapter 9 uses real ^3He parameters and shows that the one-parameter model is then wrong: it puts zero sound below first sound. The fix is the $l = 1$ parameter $F_1^s = 3(m^*/m - 1)$, which couples density to current. With it the dispersion relation becomes

$$g_3(s) = \frac{1}{\tilde{F}(s)}, \quad \tilde{F}(s) = F_0^s + \frac{F_1^s s^2}{1 + F_1^s/3}. \quad (3.7)$$

The verification script does not take this on trust: it checks it against a direct solution of the $l = 0, 1$ moment equations of the Landau kinetic equation, a 2×2 determinant with moments computed by quadrature.

The exact method carries over. With $u = 1/s^2$, $a = 1 + F_1^s/3$ and $\chi \approx P/Q$, Eq. (3.7) becomes

$$P(u) (F_0^s a u + F_1^s) - Q(u) a u = 0, \quad (3.8)$$

again a polynomial over $\mathbb{Q}$, implemented as `solve_zero_sound_root_f0_f1`. The first sound speed follows from the same parameters,

$$c_1 = v_F \sqrt{\frac{(1 + F_0^s)(1 + F_1^s/3)}{3}}. \quad (3.9)$$

Run it yourself

```
python3 simulations/qv_01_zero_sound.py      # exact QV-01 derivation
python3 verification/validate_zero_sound.py  # 60-digit check, asserts the bracket
python3 -m pytest tests/test_protocols.py -q -k "zero_sound"
```

Exercises

Exercise 3.1. Derive $a_k = 1/(2k + 1)$ from the series of artanh , and check a_1, a_2, a_3 .

Exercise 3.2. Derive Eq. (3.4) from Eq. (2.3), and show that the $M = 1$ root is admissible if and only if $F_0^s > 6/5$.

Exercise 3.3. Evaluate both edges of the bracket (3.6) at $F_0^s = 93/10$ and check that the $M = 1$ value $s = \sqrt{37/10}$ lies inside it. Why is the upper edge so loose at strong coupling?

Exercise 3.4. Show that Eq. (3.8) reduces to $u(F_0^s P - Q) = 0$ when $F_1^s = 0$. Which root does the extra factor u introduce, and why is it harmless?

Chapter 4

Kinetic theory essentials

Learning objectives

- write the spatially homogeneous Boltzmann equation and identify its collision kernel;
- use the exact Bobylev–Krook–Wu (BKW) solution for Maxwell molecules;
- define relative entropy and Fisher information and see both decrease on an exact solution.

4.1 The homogeneous Boltzmann equation

A dilute gas is described by a velocity distribution $f(v, t) \geq 0$, $v \in \mathbb{R}^d$. When f does not depend on position, it evolves by collisions alone:

$$\partial_t f = Q(f, f) = \int_{\mathbb{R}^d} \int_{S^{d-1}} B(|v - v_*|, \cos \theta) (f' f'_* - f f_*) d\sigma dv_*, \quad (4.1)$$

where (v', v'_*) are the post-collision velocities, θ is the deflection angle, and B is the *collision kernel*. For inverse-power forces $\propto r^{-s}$ the kernel factorises as $B = |v - v_*|^\gamma b(\cos \theta)$, with γ fixed by s and d . The angular part b is singular at grazing angles, and its degree of singularity ν plays a central role in Chapter 5.

Maxwell molecules are the case $\gamma = 0$. Their collision rate does not depend on the relative speed, which makes moment equations close and exact solutions possible.

4.2 Entropy and Fisher information

Let M be the Maxwellian with the same mass, momentum and energy as f . Two functionals measure the distance from equilibrium:

$$H(f | M) = \int f \ln \frac{f}{M} dv \quad (\text{relative entropy}), \quad (4.2)$$

$$I(f) = \int \frac{|\nabla_v f|^2}{f} dv \quad (\text{Fisher information}). \quad (4.3)$$

Boltzmann's H-theorem says H decreases along (4.1). Whether I also decreases is much harder. It is the subject of recent work of Imbert, Silvestre and Villani [2], reviewed in Villani's lecture notes [1], which are the subject of Chapter 5.

4.3 An exact solution: BKW

For 3D Maxwell molecules with isotropic scattering, the BKW mode [9], at unit temperature, is

$$f_K(v) = (2\pi K)^{-3/2} e^{-|v|^2/2K} \left[\frac{5K-3}{2K} + \frac{1-K}{2K^2} |v|^2 \right], \quad K(t) = 1 - (1 - K_0)e^{-t/6}, \quad (4.4)$$

in the time units of the repository's normalisation, where every particle collides at rate one. Positivity for all v requires $K \geq 3/5$; at $K = 1$ the bracket equals one and f_K is the Maxwellian.

Worked example 4.1 (Moments of BKW). Use the Gaussian moments $\langle |v|^2 \rangle_G = 3K$, $\langle |v|^4 \rangle_G = 15K^2$, $\langle |v|^6 \rangle_G = 105K^3$. Then

$$\begin{aligned} \int f_K &= \frac{5K-3}{2K} + \frac{1-K}{2K^2} 3K = 1, \\ \int |v|^2 f_K &= \frac{3(5K-3)}{2} + \frac{15(1-K)}{2} = 3, \\ \int |v|^4 f_K &= \frac{15K(5K-3)}{2} + \frac{105K(1-K)}{2} = 30K - 15K^2. \end{aligned}$$

Mass and energy are conserved exactly, and $m_4 - 15 = -15(1-K)^2$ relaxes to the Maxwellian value 15. The large simulation of Chapter 10 is judged against exactly these formulas.

4.4 Monotonicity on the exact solution

The script `validate_fisher_information_monotonicity.py` evaluates I and H along (4.4) with 30-digit quadrature, starting from $K_0 = 0.65$, strictly above the degenerate endpoint $3/5$ where I is infinite. Before trusting anything, it checks mass conservation and that the Maxwellian value $I = 3$ is recovered. It then finds both functionals decreasing monotonically: I from 3.32 to 3, and H from 0.028 to 3×10^{-11} (Figure 4.1).

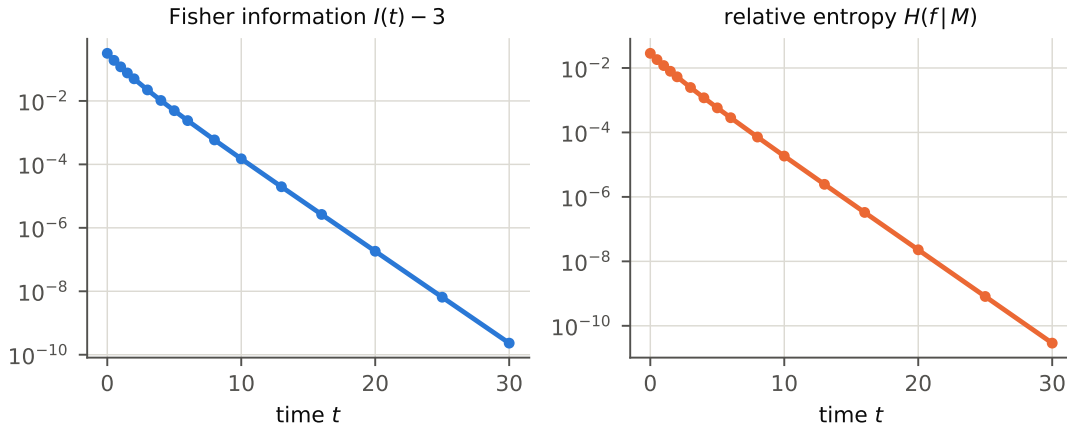


Figure 4.1: Fisher information minus its Maxwellian value, and relative entropy, along the exact BKW solution with $K_0 = 0.65$. Both decrease monotonically.

The scope of this check matters. It is an exact special case of the phenomenon Villani's notes study. It is *not* a test of Theorem 22.6, which is designed for far more singular kernels, and it makes no claim about the project's retracted `gamma_bound` formula.

Run it yourself

```
python3 verification/validate_fisher_information_monotonicity.py
```

Exercises

Exercise 4.1. Show that the bracket in Eq. (4.4) is positive for all v if and only if $K \geq 3/5$ (for $K \leq 1$).

Exercise 4.2. Compute $I(M)$ for the unit-temperature Maxwellian in d dimensions, and check the value 3 used above for $d = 3$.

Exercise 4.3. With $K_0 = 0.65$, how long does it take for $1 - K(t)$ to fall below 10^{-3} ?

Chapter 5

Villani's Theorem 22.6 and heat-kernel comparison

Learning objectives

- state Theorem 22.6 in its literal form and apply it to power-law kernels;
- understand the numerical procedure behind its worked examples, and how it was independently re-examined;
- read the $d = 4$ result as what it is: numerical evidence, conditional, offered for checking.

5.1 The statement

Theorem 22.6 of Villani's lecture notes [1] gives a sufficient condition for the Fisher information to be nonincreasing along the homogeneous Boltzmann equation. It compares the angular kernel β with a mixture β_0 of heat kernels on the sphere S^{d-1} ,

$$m \leq \frac{\beta}{\beta_0} \leq M,$$

and in its literal form concludes monotonicity if

$$\sup_{\theta} \frac{r}{B} |\partial_r B| \leq 2\sqrt{d \frac{m}{M}}. \quad (5.1)$$

For power laws, $B = |v - v_*|^\gamma b(\cos \theta)$, the left side is $|\gamma|$, and the condition becomes $|\gamma| \leq \bar{\gamma} = 2\sqrt{d m/M}$. The closer β is to a heat-kernel mixture, the closer m/M is to 1 and the larger the range of γ covered.

Lesson from the retractions: a real theorem, a wrong formula (R8)

This project once attributed the formula $\gamma_{\text{bound}} = m_r/M_r + 3/2$ to “Theorem 22.6”. The attribution was checked by fetching the source. The paper and theorem are real, and the theorem does involve m_r/M_r ; but its conclusion is the multiplicative, dimension-dependent bound (5.1), not an additive formula with a $3/2$. Neither the formula nor its value 1.9513 appears anywhere in the source. The method was renamed `kernel_regularity_bounds`, and the number is now described as a convention of this project. The two numbers 1.95 and 3.458 (below) are different objects and must never be compared.

5.2 The comparison kernels

The comparison kernels are *subordinated heat kernels*,

$$b_\omega(\theta) = \int_0^\infty \omega(t) t^{-1-\nu/2} h_t(\theta) dt,$$

where h_t is the heat kernel on S^{d-1} and ω is a weight. With $\omega \equiv 1$ one obtains the kernel of the fractional Laplacian $(-\Delta)^{\nu/2}$, which has the same grazing singularity as the collision kernel. The worked examples in the notes use tuned weights of the form $1 \mp a(1 - e^{-bt})$, found “by trial and error”, and were computed by Silvestre with Julia code [3]. The notes describe the procedure as “not a purely mathematical proof”.

5.3 Re-examination, three ways

The repository re-executes the authors’ Julia code at a pinned commit (it carries no licence, so it is run, never copied), and adds two implementations written from the mathematics, one in Python and one in Rust (`verification/theorem_22_6/`). They share no code with the Julia repository and take a different numerical route:

- the collision kernel comes from classical scattering, parametrised by $\beta = p^2/r_0^2$; it reproduces Rutherford’s cross-section to 10^{-12} in $d = 2, 3, 4$;
- each spectral time integral is an exact incomplete gamma function;
- both kernels are normalised by exact $\theta \rightarrow 0$ constants.

An exact identity anchors the comparison. At $d = 2$, $\nu = 1$ (two-dimensional Coulomb scattering), the symmetrised Rutherford kernel $1/\sin^2 \theta$ equals the symmetrised kernel of $(-\Delta)^{1/2}$ on the circle. The two independent computations agree to 6×10^{-11} , and the bound there is exactly $2 \cdot 2^{3/4}$.

Results (Level 2). Both quoted claims reproduce. The minimum of the bound over the tabulated ν is 4.355 in $d = 3$ (claim ≥ 4.3 ; authors’ code 4.3568) and 3.364 in $d = 2$ (claim > 3.3 ; authors’ code 3.3630). Rust agrees with Python to 2×10^{-9} . One detail differs: the authors’ adaptive quadrature underestimates Λ_b as $\nu \rightarrow 2$, where the integrand tends to t^{-1} . At $d = 2$, $\nu = 1.999$ it gives 3.932 against the exact 3.997, obtained from

$$\int_0^\infty t^{-1-s} (e^{-\alpha t} - e^{-\beta t}) dt = \Gamma(-s) (\alpha^s - \beta^s). \quad (5.2)$$

The error is conservative: the tabulated bounds are slightly low.

Denser sampling (E1). An infimum sampled on a grid can only be overestimated. With 52 values of ν and up to 400 angles both claims survive, and the eleven-angle grid overestimates the bound by at most 0.021. In $d = 2$ a dip appears between the tabulated $\nu = 1.0$ and 1.25: the minimum is 3.3593 at $\nu \approx 1.06$, still above 3.3 (Figure 5.1).

The weights (E2). A grid search over the two-parameter families containing the notes’ weights finds them within 0.06 of the best in their family. With the plain fractional Laplacian, both claims fail over most of their range (Figure 5.2).

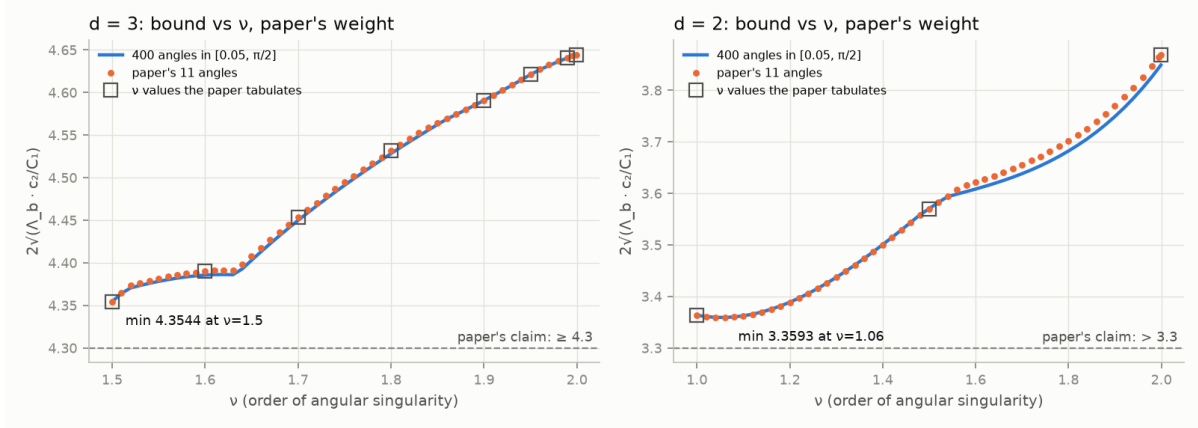


Figure 5.1: Experiment E1: the bound against ν with the notes' weights, on 400 angles (line) and the notes' eleven (dots). Dashed: the claimed thresholds.

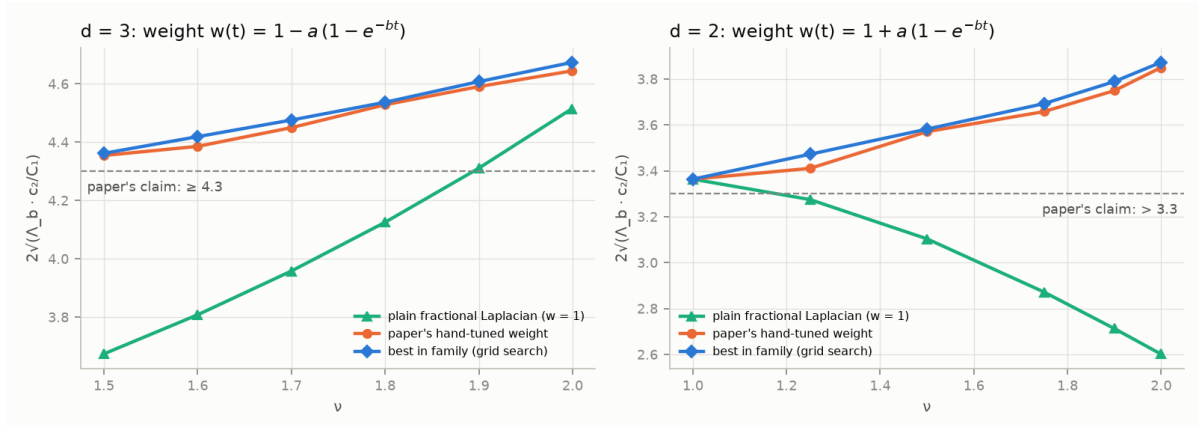


Figure 5.2: Experiment E2: plain fractional Laplacian, the notes' weight, and the best weight in its family.

5.4 Applying the theorem to a model kernel (E3)

The project's phenomenological roton kernel is $\beta(\cos \theta) = \frac{1}{2}(1 + \cos^2 \theta) e^{-(1 - \cos \theta)/10}$, an analytic model chosen for its forward-peaked shape, with no measured data in it. After symmetrisation it is close to $\frac{4}{3} + \frac{2}{3}P_2(\cos \theta)$, which a single heat kernel matches at $t = \ln(10)/6 \approx 0.384$. A linear program over finite heat-kernel mixtures finds $m/M = 0.99654$, with the optimiser landing at $t = 0.3846$ unprompted. In $d = 3$ this gives

$$\bar{\gamma} = 2\sqrt{3 \times 0.99654} = 3.458,$$

close to the ceiling $2\sqrt{3}$ (Figure 5.3). This is a property of the model kernel, not of real rotons.

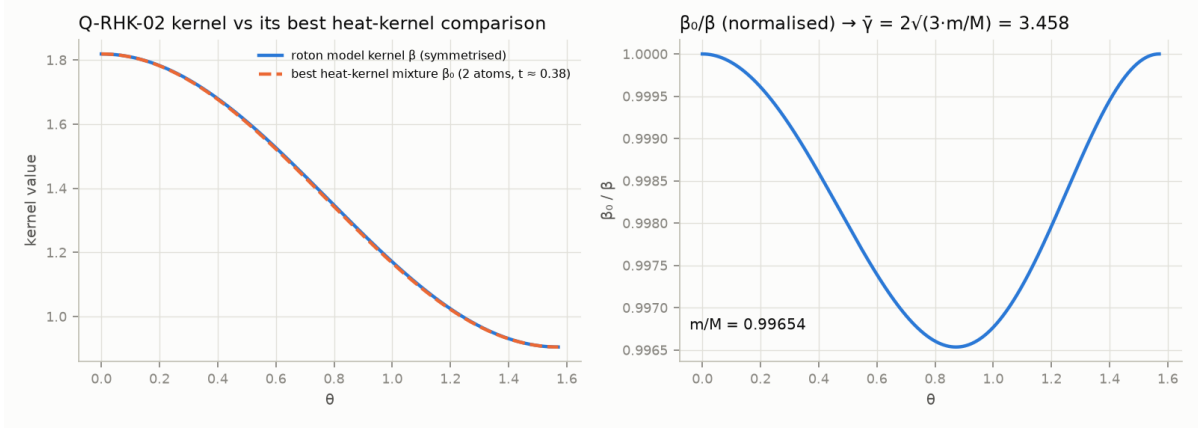


Figure 5.3: Experiment E3: the model kernel and its best heat-kernel comparison (left); their ratio, with $m/M = 0.99654$ (right).

5.5 The open case in dimension four (E4)

Remark 22.8 of the notes states that $\gamma \in (-3, -2\sqrt{2}]$ in $d = 4$ is not covered. Classical scattering by a force $\propto r^{-s}$ in dimension d gives

$$\nu = \frac{d-1}{s-1}, \quad \gamma = 1 - 2\nu,$$

so the open range is $\nu \in [1.9142, 2)$, and the literal criterion requires $m/M > ((2\nu - 1)/4)^2$, at most $9/16$. A linear program over three admissible families of comparison kernels (tempered singular kernels with weight $t^{-1-\nu/2}e^{-\mu t}$, bounded kernels with weight $t^{-1-\nu/2}(1 - e^{-bt})$, and pure heat kernels on S^3) finds $m/M \geq 0.996$ at sixteen sampled ν , re-checked on 800 angles down to $\theta = 0.01$. Hence $\bar{\gamma} \approx 3.99 > |\gamma|$. The result survives halving the basis (0.9954) and using the tempered family alone (0.9928). *It fails without tempering: m/M drops to 0.48, below the required $9/16$.*

How to read the $d = 4$ result

This is numerical evidence at the standard of the notes' own worked examples, not a proof. It assumes Theorem 22.6 holds as stated in $d = 4$, and below $\theta = 0.01$ it relies on the exact $\theta \rightarrow 0$ limit. It is offered to the authors for checking. Along the way a numerical artefact appeared: one spectral cut-off chosen for the smallest angle lost accuracy near $\pi/2$ in $d = 4$ and briefly produced $m/M = 0.937$. The fix was a per-block cut-off. The general lesson: when a spectral sum cancels, measure its error against the *smallest* kernel value on the grid.

Run it yourself

```
python3 verification/theorem_22_6/reproduce.py      # Python
(cd verification/theorem_22_6/rust && cargo run -release) # Rust
verification/theorem_22_6/run.sh                  # authors' Julia code
python3 -m pytest tests/test_theorem_22_6_kernels.py -q
```

The experiments E1–E4b and their run times are listed in `verification/theorem_22_6/EXPERIMENTS.md`.

Exercises

Exercise 5.1. Show that $\gamma \in (-3, -2\sqrt{2}]$ corresponds to $\nu \in [(1 + 2\sqrt{2})/2, 2) \approx [1.9142, 2)$, and that the literal criterion in $d = 4$ requires $m/M \geq (2\nu - 1)^2/16$.

Exercise 5.2. Compute $\bar{\gamma}$ in $d = 3$ for $m/M = 0.99654$, and the ceiling as $m/M \rightarrow 1$.

Exercise 5.3. Prove Eq. (5.2) for $0 < s < 1$ and $\alpha, \beta > 0$. Why does naive quadrature struggle as $s \rightarrow 1$ in the application above?

Exercise 5.4. Explain why sampling the ratio β/β_0 on a finite set of angles can only *overestimate* m/M , and therefore the bound.

Chapter 6

Phase mixing and a second-order response

Learning objectives

- compute a second-order Volterra response exactly by Cauchy products;
- identify the result with a closed form;
- explain phase mixing for free transport, and why a Taylor series at $t = 0$ cannot contain a plasma echo.

6.1 The protocol

The protocol QVE-02 takes a linear density response $\rho^{(1)}(t) = \text{sinc } t = \sin t/t$, forms the field $E^{(1)} = \int_0^t \rho^{(1)}$, the source $S^{(2)} = \rho^{(1)}E^{(1)}$ by an exact Cauchy product of Taylor series, and one further integration. The engine returns

$$\rho^{(2)}(t) = \frac{1}{2}t^2 - \frac{1}{18}t^4 + \frac{13}{4050}t^6 - \frac{4}{33075}t^8 + \frac{73}{22325625}t^{10} + \mathcal{O}(t^{12}). \quad (6.1)$$

The arithmetic is exact and correct. And $E^{(1)} = \text{Si}(t)$, the sine integral, so $\rho^{(2)} = \int \rho^{(1)}E^{(1)} = \text{Si}(t)^2/2$. Equation (6.1) is simply the Taylor series of $\text{Si}(t)^2/2$, and the test suite now asserts this term by term.

Lesson from the retractions: the echo that was not (R2)

Earlier versions presented Eq. (6.1) as the discovery of a nonlinear plasma echo, and a stored file was labelled “True 1D1V Vlasov–Poisson phase space simulation”, although no code in the repository produced it. An echo is a large-time phenomenon: two pulses with distinct wavenumbers k_1, k_2 , applied at 0 and τ , produce a macroscopic response near $t = \tau k_2/(k_2 - k_1)$, long after each linear response has phase-mixed away. A Taylor expansion about $t = 0$ with one input and no phase space cannot contain that. The claim is retracted, and the protocol is kept as what it is: an exact second-order Volterra response, tested against its closed form so the identification cannot be lost again.

6.2 What the input really is

The positive reading, now proved, concerns the input. Free transport $\partial_t f + v \partial_x f = 0$ is solved by $f(x, v, t) = f_0(x - vt, v)$. For a perturbation $e^{ikx}g(v)$, the density mode is

$$\rho_k(t) = \int e^{-ikvt} g(v) dv,$$

the Fourier transform of g at kt . For every integrable g it tends to zero (Riemann–Lebesgue). This is *phase mixing*: nothing is dissipated, but the density perturbation disappears as the distribution filaments in velocity.

Proposition 6.1 (`flat_top_mode`, `flat_top_mode_tendsto_zero` in `PhaseMixingLink.lean`). For the flat-top distribution $g = \frac{1}{2}\mathbb{1}_{[-1,1]}$ and $a \neq 0$, $\frac{1}{2} \int_{-1}^1 \cos(av) dv = \sin a/a$. Hence the density mode is $\sin(kt)/(kt)$, which equals QVE-02's input at $k = 1$, and $\sin t/t \rightarrow 0$ as $t \rightarrow \infty$, bounded by $1/t$.

By Archimedes' hat-box theorem, the one-dimensional marginal of the uniform measure on the sphere S^2 is exactly this flat top, so $\rho^{(1)}$ is also the free-streaming mode of a uniform Fermi surface; the test suite checks this numerically. The decay is algebraic, like $1/t$, in contrast with the Gaussian $e^{-k^2 t^2/2}$ of a Maxwellian. The second-order term tends to $\text{Si}(\infty)^2/2 = \pi^2/8$: it approaches a constant, and nothing echoes.

Run it yourself

```
python3 -m pytest tests/test_protocols.py -q -k qve_02
python3 -m pytest tests/test_qve02_phase_mixing.py -q
```

Exercises

Exercise 6.1. Using $\text{Si}(t) = t - t^3/18 + t^5/600 - \dots$, compute the first three coefficients of $\text{Si}(t)^2/2$ and compare with Eq. (6.1).

Exercise 6.2. Show that the density mode of the unit Maxwellian $g(v) = (2\pi)^{-1/2}e^{-v^2/2}$ is $e^{-k^2 t^2/2}$.

Exercise 6.3. Show that if a point is uniform on the unit sphere S^2 , its z -coordinate is uniform on $[-1, 1]$ (Archimedes).

Part II

Practice

Chapter 7

The engine

Learning objectives

- find your way around the repository;
- know which stage computes what, and which test protects it;
- run the full verification suite.

7.1 Layout

Path	Contents
agora_swarm/	the exact engine: stages and orchestrator
tests/	the pytest suite (49 tests)
verification/	independent numerical checks, data confrontations, Theorem 22.6 experiments
simulations/	protocol drivers and the large-scale simulation
lean4_formalization/	Lean 4 library AgoraPhysics
alexandrie_data/	committed results (small JSON files)
notebooks/	Jupyter notebooks for students
agora_mcp/	local MCP server (Chapter 11)
docs/	papers, this book, figures

The top-level Markdown files are part of the scientific record: `PROTOCOL_REGISTRY.md` (what each protocol does), `RETRACTIONS.md` (what has been withdrawn), `ROADMAP.md` (what is open), and `verification/THEORY_EXPERIMENT_LINKS.md` (which experiment tests which theorem).

7.2 Three stages

The engine is a pipeline. Earlier versions named its stages after living scientists, which wrongly implied their involvement (R1); they are now named for what they do.

LinearResponseStage produces exact rational coefficient sequences: the zero-sound kernel, the legacy kernel g , the linear density perturbation, the model roton kernel.

KineticStage performs the kinetic algebra: `pade_diagonal`, `solve_zero_sound_root`, `solve_zero_sound_root_f0_f1`, `compute_second_order_volterra_response` and `kernel_regularity_bounds`.

Orchestrator (`agora_swarm/orchestrator.py`) sequences the protocols, enforces the exactness rule at stage boundaries, and writes results as exact strings to the *Alexandrie* vault, `alexandrie_data/`.

7.3 The protocols

ID	What it computes	Status
QV-01	zero-sound roots by exact Padé	works at strong coupling; negative result at weak coupling; extended to (F_0^s, F_1^s)
QVE-02	second-order Volterra response	exact; equals $\text{Si}(t)^2/2$; “echo” retracted
Q-RHK-02	bounds for a model roton kernel	1.9513 is a project convention (R8); Theorem 22.6 gives $\bar{\gamma} = 3.458$
Q-RIP-03	$L_* = 2d$ at $d = 2$	definitional ($2 \cdot 2 = 4$), labelled a conjecture (R3)

7.4 Tests that defend the claims

Two tests are adversarial by design. One re-derives the kernel from its closed form rather than comparing it with a table; the other requires float input to raise. Others pin behaviour that might otherwise be “fixed” into a lie: `test_zero_sound_weak_coupling_has_no_admissible_root` asserts that the engine keeps reporting *no root* at weak coupling. A test suite that only checks success cannot protect a negative result; this one does.

Run it yourself

```
python3 -m pytest tests/ -q
python3 verification/validate_zero_sound.py
python3 verification/validate_kernel_regularity_bounds.py
python3 verification/validate_fisher_information_monotonicity.py
python3 verification/theorem_22_6/reproduce.py
python3 verification/helium_kinematics_data.py          # downloads from arXiv
python3 verification/he3_landau/extract_table_ix.py     # downloads from arXiv
python3 verification/he3_landau/zero_sound_real_he3.py
```

These are the same commands continuous integration runs (`.github/workflows/python_tests.yml`).

Exercises

Exercise 7.1. Write a pytest test asserting that `solve_zero_sound_root` at $M = 1$ returns $u = 15/(9 + 5F)$ for $F = 7/2$, and that it refuses the float 3.5.

Exercise 7.2. Why would a well-meaning contributor be tempted to delete the weak-coupling test, and what would be lost?

Chapter 8

Lean 4 for physicists

Learning objectives

- read a short Mathlib proof about real functions;
- audit a Lean development with `#print axioms`;
- state precisely what a Lean theorem certifies in this project.

8.1 Setting up

The library `AgoraPhysics` lives in `lean4_formalization/`. It pins the toolchain `lean-prover/lean4:v4.31.0` and Mathlib at revision `ec6e08b4`. Always fetch Mathlib’s compiled cache: compiling Mathlib from source takes hours.

Run it yourself

```
cd lean4_formalization
lake exe cache get
lake build
```

The default target is the library, not the executable: building the executable would force native compilation of all of Mathlib just to link a binary that prints a greeting.

8.2 Reading a proof: the upper edge

Here is the upper edge of the zero-sound bracket, verbatim from `ZeroSoundBracket.lean`:

```
noncomputable def g3 (s : ℝ) : ℝ := s / 2 * log ((s + 1) / (s - 1)) - 1

theorem root_upper {F s : ℝ} (hF : 0 < F) (hs : 1 < s) (hroot : g3 s = 1 / F) : s -
  1 ≤ F := by
  have h1 : 0 < s - 1 := by linarith
  have h := g3_le hs
  rw [hroot] at h
  exact (one_div_le_one_div hF h1).mp h
```

Read it as a physicist would.

1. `g3` is $\chi(s)$ as a function of s , over the real numbers.
2. The hypotheses are $F > 0$, $s > 1$ (undamped) and that s is a root.
3. `g3_le` is the lemma $g_3(s) \leq 1/(s - 1)$, from $\ln y \leq y - 1$.

4. Rewriting with the root equation turns it into $1/F \leq 1/(s-1)$.
5. Since both sides are positive, inverting reverses the inequality: $s-1 \leq F$.

Each tactic is a small, checkable step: `linarith` solves linear arithmetic, `rw` rewrites with an equation, and `one_div_le_one_div` is a Mathlib lemma.

The lower edge, `root_lower`, follows the same pattern with the bound $g_3(s) \geq \frac{1}{2} \ln \frac{2}{s-1} - 1$ and then exponentiates. Its docstring notes that it does not need $F > 0$: `g3_pos` rules out a root for $F < 0$, and in Lean $1/0 = 0$ rules it out for $F = 0$. Details like this are what a proof assistant forces you to face.

8.3 A proof about phase mixing

```
theorem flat_top_mode_tendsto_zero : Tendsto (fun t : ℝ => sin t / t) atTop (N 0)
:= by
  have hinv : Tendsto (fun t : ℝ => t⁻¹) atTop (N 0) := tendsto_inv_atTop_zero
  have hbound : ∀ᶜ t : ℝ in atTop, ||sin t / t|| ≤ t⁻¹ := by
    filter_upwards [eventually_gt_atTop 0] with t ht
    rw [Real.norm_eq_abs, abs_div, abs_of_pos ht, div_eq_mul_inv]
    exact mul_le_of_le_one_left (inv_nonneg.mpr ht.le) (abs_sin_le_one t)
  exact squeeze_zero_norm' hbound hinv
```

The structure is the textbook squeeze argument: $1/t \rightarrow 0$, and $|\sin t/t| \leq 1/t$ eventually, so $\sin t/t \rightarrow 0$. `Tendsto ... atTop (N 0)` is Mathlib's way of writing $\lim_{t \rightarrow \infty} = 0$ with filters.

8.4 The axiom audit

Every Lean file of the project ends with lines such as

```
#print axioms AgoraPhysics.ZeroSoundBracket.zero_sound_bracket
```

The build log then lists the axioms each theorem depends on. Only the three standard axioms of Mathlib are acceptable: `propext`, `Classical.choice` and `Quot.sound`. If `sorryAx` appears, a proof is incomplete somewhere upstream, even if the file itself contains no `sorry`. Continuous integration fails in that case.

Lesson from the development: a proof that silently was not

While writing `PhaseMixingLink.lean`, a `simp` step failed to close its goal in the expected shape. Lean elaborated the theorem with `sorryAx` standing in for the failed step, and it was the axiom audit that flagged it. Restating the bound in norm form fixed it. A green build is not a proof; an axiom list is evidence.

8.5 What the Lean library certifies

- `Protocols.lean`: arithmetic identities, such as the $[1/1]$ order condition $a_2 + q_1 a_1 = 0$ and the fact that $u = 10/37$ is a root of $Q - \frac{93}{10}P$. Two theorems there are labelled vacuous in the source itself: one proves $2 \cdot 2 = 4$ after unfolding a definition, one has its hypothesis as its conclusion (R3). They are kept, labelled, as a record.
- `ZeroSoundBracket.lean`: real analysis about the Landau zero-sound relation (Theorem 3.1).
- `PhaseMixingLink.lean`: real analysis about free transport.

None of these certifies anything about real ^3He or ^4He . That connection, where it is made, is made by the data comparisons of Chapter 9. Lemmas `g3_pos`, `g3_le` and `g3_ge` are restated, with attribution, from a companion project, `SocrateAI-Scientific-QuantumFluids`, which pins a different Lean version.

Exercises

Exercise 8.1. Explain in words why `root_lower` needs no hypothesis $F > 0$, including the case $F = 0$.

Exercise 8.2. The theorem `admissible_singularity_limit` (`h :  $\gamma < \text{gamma\_bound}$`) : $\gamma < 16063/8232$ with `gamma_bound := 16063/8232` is proved by `exact h`. What does it establish?

Chapter 9

Confronting models with data

Learning objectives

- derive Landau parameters of real ^3He from published thermodynamic tables;
- see a mathematically correct model fail on real parameters, and repair it;
- test machine-checked kinematic statements on open neutron-scattering data for ^4He .

9.1 Getting data honestly

Two rules govern data in this project. Every value is fetched from a named source by code, with a checksum where possible, and never typed from memory. And third-party files are not redistributed: the scripts download them to a cache.

Lesson from the retractions: “no dataset exists” (R9)

After a search of Zenodo, Hugging Face, Materials Cloud and the ILL portal, the project once concluded that no open data existed for the roton regime of ^4He . That was wrong: the authors of Godfrin *et al.* 2021 [4] had published their dispersion data as arXiv *ancillary files*, and the search had not looked there. A negative search result is a claim about the search, not about the world.

9.2 Real ^3He : Landau parameters from thermodynamics

Kollar and Vollhardt [7] tabulate, from Greywall’s measurements [8], the molar volume V , the specific-heat coefficient γ/R and the compressibility κ of normal liquid ^3He at $T \rightarrow 0$, from 0 to 29 bar (their Table IX). The script `extract_table_ix.py` fetches their arXiv PDF, reads page 19 from its *text layer* (no OCR), keeps each value exactly as printed, and checks the table internally:

- the $\partial\gamma/\partial P$ column against finite differences of γ , to 2.4×10^{-3} ;
- κ against $-V^{-1}\partial V/\partial P$, to 6.8×10^{-3} ;
- the prefactor of their Eq. (30) recomputed from CODATA constants, 3.2869×10^{-4} against the printed 3.285×10^{-4} .

From γ follows the effective mass, $m^*/m = (2/\pi^2)(\gamma/R)T_F$, with T_F the bare Fermi temperature at that density; then $F_1^s = 3(m^*/m - 1)$. From κ and their Eq. (30) follows F_0^s . At 0 bar: $V = 36.846 \text{ cm}^3$, $\gamma/R = 2.7411$, $T_F = 4.957 \text{ K}$, $m^*/m = 2.753$, $F_1^s = 5.260$, $F_0^s = 10.28$, $v_F = 60.04 \text{ m/s}$.

9.3 The one-parameter model fails

Table 9.1 and Figure 9.1 show the outcome (Level 4). With F_0^s alone, the model predicts zero sound *slower* than first sound at every pressure. In ^3He zero sound is slightly faster. The model is exact as mathematics and wrong as physics, and only real parameters could show it. Adding F_1^s restores the observed ordering, with $c_0/c_1 = 1.036$ at 0 bar falling to 1.005 at 29 bar. First sound computed from Landau theory, Eq. (3.9), agrees with the thermodynamic $1/\sqrt{\rho\kappa}$ from the same table to 2×10^{-10} , and the exact $[4/4]$ root of Eq. (3.8) agrees with a 60-digit reference to better than 10^{-8} at 0 and 29 bar.

Table 9.1: Zero sound c_0 and first sound c_1 (m/s) in liquid ^3He from the Landau parameters of [7] (`alexandrie_data/HE3-LANDAU/zero_sound_real_he3.json`).

P (bar)	F_0^s	F_1^s	c_0, F_0^s only	$c_0, (F_0^s, F_1^s)$	c_1	c_0/c_1
0	10.28	5.26	120.8	200.1	193.2	1.036
6	21.43	7.29	140.5	260.1	255.5	1.018
15	41.95	9.69	161.9	332.9	329.9	1.009
29	73.94	13.00	174.4	402.7	400.6	1.005

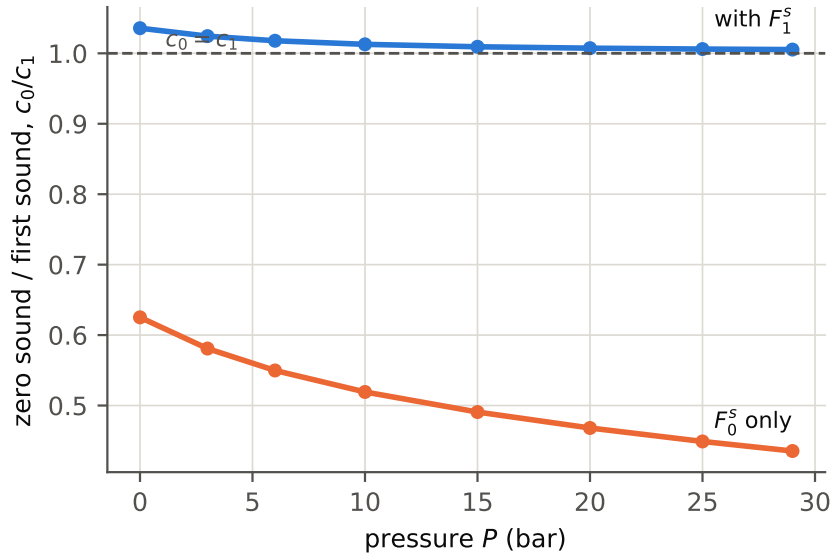


Figure 9.1: Ratio of zero sound to first sound for real ^3He , all eight computed pressures. With F_0^s alone the ratio is below one, which is wrong.

One caveat: the absolute $c_1 = 193$ m/s at 0 bar is about 5% above commonly quoted measurements, and the source notes that its compressibility is least certain below about 2 bar. Ratios are more reliable here than absolute speeds. The model also neglects F_2^s and higher, finite temperature and collisions.

9.4 Real ^4He : neutron data and machine-checked kinematics

Superfluid ^4He has a single excitation branch $\varepsilon(Q)$: phonons at small Q , a local maximum (the *maxon*) and a local minimum (the *roton*, with gap Δ). A companion project formalises kinematic statements about such a branch. The script `helium_kinematics_data.py` tests them on the IN5 dispersion data of [4]: seven pressures from 0 to 24.08 bar, with per-point

uncertainties. The files are fetched from arXiv and checked by SHA-256. The fit reproduces the published sound velocity (1.5685 against 1.5679 meV Å) and roton gap (0.7417 against 0.7445 meV).

Maxon versus the roton-pair threshold. A maxon above 2Δ can decay into two rotons. The data show the maxon below 2Δ up to 10 bar and above it at 24.08 bar, by $+0.0106 \pm 0.0005$ meV, about 21σ (Figure 9.2), as Beauvois *et al.* reported [5]. A proved lemma adds something the data alone cannot: whether an energy exceeds 2Δ cannot be changed by a proportional recalibration of the energy scale, so no such calibration error can reverse the conclusion.

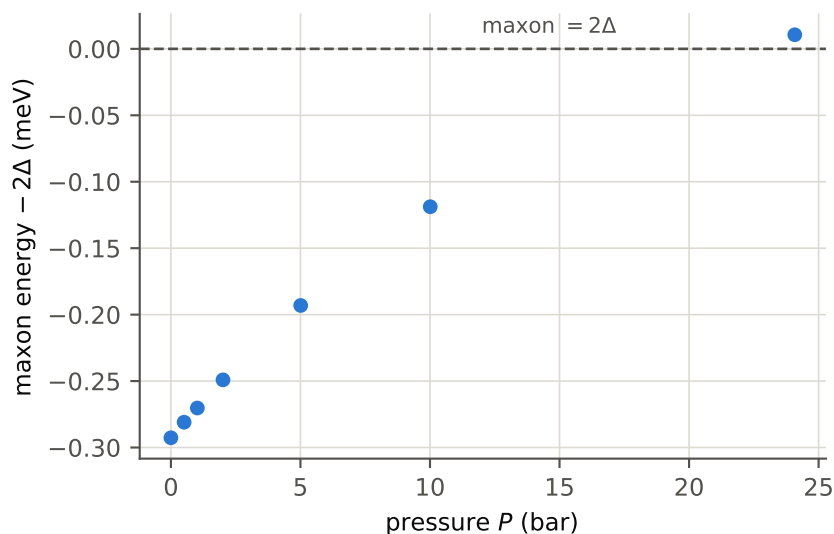


Figure 9.2: Maxon energy minus twice the roton gap, from the IN5 dispersion of [4] (alexandrie_data/HE4-KIN/helium_kinematics.json). Error bars are one standard deviation.

Three-phonon decay. For $\varepsilon = cQ(1 + aQ^2)$, a proved equivalence says that three-phonon decay is open if and only if $a \geq 0$. At saturated vapour pressure a fit on $Q \leq 0.15 \text{ Å}^{-1}$ gives $a = +0.95 \pm 0.25 \text{ Å}^2$, upward curvature at 3.8σ , so decay is open. The sign change reported near 18–20 bar cannot be decided from these files, which start at $Q = 0.15 \text{ Å}^{-1}$ for $P > 0$.

Lesson from the development: a test that asked too much

The first version of this test asserted the 18–20 bar sign change, which the data cannot resolve, and required 2σ significance on every fit range. It was redesigned to report “undecided” where the data are silent and to treat a non-significant result as “not contradicted” rather than as a failure. The change is recorded in the repository, not hidden.

Run it yourself

```
python3 verification/he3_landau/extract_table_ix.py
python3 verification/he3_landau/zero_sound_real_he3.py
python3 verification/helium_kinematics_data.py
```

Exercises

Exercise 9.1. From $\gamma/R = 2.7411$ and $T_F = 4.9569$ K at 0 bar, compute m^*/m and F_1^s .

Exercise 9.2. With $v_F = 60.042$ m/s, $F_0^s = 10.2788$ and $F_1^s = 5.2601$, compute c_1 from Eq. (3.9).

Exercise 9.3. Prove the recalibration lemma: if every energy is multiplied by the same $\lambda > 0$, the sign of $\varepsilon_{\text{maxon}} - 2\Delta$ is unchanged.

Exercise 9.4. The excess at 24.08 bar is 0.0106 ± 0.0005 meV. How many standard deviations is that, and what assumption does “ 21σ ” rely on?

Chapter 10

A large simulation certified by an exact solution

Learning objectives

- simulate the homogeneous Boltzmann equation with a stochastic particle method (DSMC);
- judge a floating-point simulation against an exact solution, with no fitted parameter;
- recognise the Monte Carlo error law $N^{-1/2}$ and a discretisation bias.

10.1 The idea

The project's rule is that an exact result certifies a floating-point computation, never the reverse. The large simulation applies that rule at scale. The floating-point computation is a Direct Simulation Monte Carlo (DSMC) method of Nanbu–Bird type with up to 10^6 particles; the exact result is the BKW solution of Chapter 4. The simulation is judged against the exact curves, and its error must shrink like $N^{-1/2}$. Nothing is fitted.

10.2 The model

The script `simulations/large_scale/dsmc_bkw.py` treats 3D Maxwell molecules with isotropic scattering, normalised so that every particle collides at rate one:

$$\partial_t f = Q^+(f, f) - f, \quad Q^+(f, f)(v) = \iint f(v') f(v'_*) dv_* \frac{d\omega}{4\pi}.$$

A collision of v and w keeps the centre of mass $V = (v+w)/2$ and the relative speed $|u| = |v-w|$, and sends

$$v' = V + \frac{1}{2}|u|\omega, \quad w' = V - \frac{1}{2}|u|\omega, \quad \omega \text{ uniform on } S^2.$$

10.3 The exact benchmark

Averaging one collision over ω and over independent, isotropic partners gives a closed equation for the fourth moment $m_4 = \mathbb{E}|v|^4$:

$$\frac{dm_4}{dt} = -\frac{1}{3} \left(m_4 - \frac{5}{3} m_2^2 \right), \quad (10.1)$$

so m_4 relaxes at rate exactly $1/3$ toward 15 when $m_2 = 3$. For BKW, $m_4 - 15 = -15(1-K)^2$ (Chapter 4), and the moment equation forces $(1-K)^2 \propto e^{-t/3}$, that is $K(t) = 1 - (1-K_0)e^{-t/6}$.

The time scale is *derived* for this normalisation, not fitted. The script also uses $m_6 = 315K^2 - 210K^3$, the exact marginal distribution of v_x , and the relative entropy computed by quadrature.

10.4 The algorithm

Collisions form a Poisson process with total rate $N/2$. Each step of length Δt draws $k \sim \text{Poisson}(N\Delta t/2)$ and collides k disjoint random pairs. The only approximation is disjointness within a step: the per-step contraction of $m_4 - 15$ becomes $1 - \Delta t/3$ instead of $e^{-\Delta t/3}$. The script computes the effect of this on m_4 in closed form and reports it. In the committed run, with $\Delta t = 10^{-2}$, it is at most 1.0×10^{-4} relative on m_4 , below the Monte Carlo error even at $N = 10^6$. The cost is proportional to the number of collisions, $\mathcal{O}(NT)$, independent of Δt .

The default initial condition is $K_0 = 3/5$, the extreme member of the BKW family, where f vanishes at $v = 0$. The initial particles are sampled from that exact f .

10.5 What to look for

- **Moments.** $m_4(t)$ and $m_6(t)$ from particles should follow the exact curves within statistical error bars.
- **Error law.** Repeating with $N = 10^4, 10^5, 10^6$ and several seeds, the error should fall like $N^{-1/2}$: a factor of about 3.2 per decade of N . A slower decay would reveal a bias, for example from the time step.
- **Entropy.** The relative entropy is estimated from a histogram of speeds. Its exact counterpart is the same binned divergence of the exact f ; binning only lowers it (data-processing inequality). Both must decrease.

10.6 Results

The committed summary `alexandrie_data/SIMULATIONS/dsmc_bkw_summary.json` records the maintainers' run to $t = 30$ from $K_0 = 3/5$ (Table 10.1). The root-mean-square relative error of m_4 over the trajectory falls from 6.8×10^{-3} at $N = 10^4$ to 6.2×10^{-4} at $N = 10^6$, and a fit of $\log(\text{error})$ against $\log N$ gives slope -0.519 , against the Monte Carlo prediction $-1/2$. Energy is conserved to rounding ($\sim 10^{-15}$). The exact relative entropy decreases strictly, from 0.0639 to 4.9×10^{-11} , and the particle estimate follows it with an error that also shrinks with N . Nothing was fitted: the only inputs are the collision rule and the initial f . The numbers were current when this book was written; the notebook regenerates them. Large raw output is written to a cache directory, not to git.

Table 10.1: DSMC against the exact BKW solution: root-mean-square errors over $0 \leq t \leq 30$, averaged over seeds (`dsmc_bkw_summary.json`).

N	seeds	rel. error m_4	rel. error m_6	abs. error H	wall time (s)
10^4	16	6.8×10^{-3}	2.2×10^{-2}	2.1×10^{-3}	5
10^5	4	2.0×10^{-3}	6.4×10^{-3}	3.1×10^{-4}	17
10^6	2	6.2×10^{-4}	2.1×10^{-3}	4.8×10^{-5}	135

Run it yourself

```
python3 simulations/large_scale/dsmc_bkw.py -quick      # < 30 s, as in CI
python3 simulations/large_scale/dsmc_bkw.py            # N = 1e4, 1e5, 1e6, a few minutes
python3 simulations/large_scale/he3_pressure_sweep.py  # exact solver over all pressures
jupyter notebook notebooks/04_large_simulation_dsmc_vs_exact_bkw.ipynb
```

The notebooks 01_exact_zero_sound_quickstart, 02_real_he3_zero_sound and 03_theorem_22_6_numerics accompany Chapters 3, 9 and 5.

Exercises

Exercise 10.1. Check that $m_4 = 30K - 15K^2$ satisfies the moment equation when $K(t) = 1 - (1 - K_0)e^{-t/6}$.

Exercise 10.2. With $N = 10^6$ particles, estimate the standard error of the particle average of $|v|^4$ at equilibrium, given that $\text{Var}(|v|^4) = \mathbb{E}|v|^8 - 15^2$ and $\mathbb{E}|v|^8 = 945$ for the unit Maxwellian.

Exercise 10.3. Over a time T with step Δt , compare $(1 - \Delta t/3)^{T/\Delta t}$ with $e^{-T/3}$ for $T = 30$, $\Delta t = 10^{-2}$. Why can this bias be computed exactly rather than estimated?

Chapter 11

The MCP server: the engine as a tool for AI agents

Learning objectives

- start the local MCP server and connect a client to it;
- know what each tool computes and what it refuses.

The Model Context Protocol (MCP) lets an AI assistant call tools provided by a local program. The package `agora_mcp` exposes the engine as such a server, over standard input and output, on the same machine. It is started with

```
python3 -m agora_mcp
```

and the repository's `.mcp.json` registers it for Claude Code as `agora-physics`. Full documentation is in `docs/MCP.md`.

Tool	What it does
<code>zero_sound_root</code>	exact Padé root for (F_0^s, F_1^s) at order M
<code>pade_approximant</code>	exact $[M/M]$ approximant of the zero-sound kernel
<code>he3_landau_parameters</code>	real ^3He parameters at a tabulated pressure, with provenance
<code>helium_kinematics</code>	committed results of the ^4He data tests
<code>theorem_22_6_bound</code>	kernel comparison behind Theorem 22.6's worked examples
<code>list_protocols</code>	the protocols and the whitelisted verification scripts
<code>get_retractions</code>	the entries of <code>RETRACTIONS.md</code>
<code>run_verification</code>	runs one whitelisted verification script

The server applies the same rules as the engine. Numbers are sent as strings or fractions ("93/10"); a JSON float is refused with an explanation, for the reason of Chapter 1. The data tools serve only tabulated values, without interpolation, and return their source. `run_verification` runs only whitelisted scripts, and scripts that download data run only if the server was started with `AGORA_MCP_ALLOW_NETWORK=1`.

Run it yourself

In Claude Code, from the repository root, the server is picked up from `.mcp.json`. Then ask, for example: "Use `zero_sound_root` with $F_0s = 10.279$ and $F_1s = 5.260$ at $M = 4$, and compare with the first sound in `he3_landau_parameters` at 0 bar."

Exercises

Exercise 11.1. Why does the server refuse the JSON number `9.3` but accept the string `"9.3"`? What exact value does the string become?

Chapter 12

Contributing as a scientist or an AI agent

Learning objectives

- choose the right channel for a contribution;
- write a numeric-calculation request that an agent can execute;
- prepare a pull request that meets the project's standard.

12.1 Channels

The repository's `CONTRIBUTING.md` describes the process and `AGENTS.md` the rules. In brief, there are four kinds of contribution.

1. **Numeric calculation request.** Open an issue with the form of that name. Give exact inputs (as rationals where possible), the quantity wanted, the precision or evidence level needed, and the source of any physical input. Fully specified requests are labelled `agent-ready`.
2. **Roadmap proposal.** Propose a protocol, a dataset to confront, a theorem to formalise or a method to replace. State what would count as success *and* what would count as a negative result.
3. **Scientific error or retraction.** If a claim is wrong, overstated or mis-attributed, report it. This is the most valuable contribution. Confirmed errors enter `RETRACTIONS.md` with credit, and are fixed everywhere the claim appears.
4. **Code, proofs, data, documents** through pull requests.

12.2 Rules for everyone, human or agent

The rules in `AGENTS.md` are the ones this book has motivated chapter by chapter:

- no floats in derivations (Chapter 1);
- no numbers from memory: fetch from a named source, in code (Chapter 9);
- label every claim with its evidence level, and never call numerical evidence a proof;
- read `RETRACTIONS.md` first, never re-introduce a retracted claim, and when a claim is withdrawn, search the *whole* repository for it;

- Lean proofs audited with `#print axioms`, not `grep` (Chapter 8);
- negative results are results, and are tested (Chapter 3);
- no person-named agents, no implied endorsement, no copying of unlicensed code;
- no secrets in code, logs or commits; big data stays out of git.

Lesson from the history: fixes that stopped at one file

Several retractions were first fixed in one place but not in others that repeated the claim: the “echo” framing survived in the orchestrator’s printed output and in a Lean theorem name after it had been removed from the Python stage. When a claim is withdrawn, `grep` the code, the orchestrator, the persisted JSON, the Lean names and comments, and the LaTeX.

12.3 AI agents

Agents are first-class contributors under the same rules. Claude Code reads the repository’s `CLAUDE.md`, which imports `AGENTS.md`, and can use the local MCP server. Google Jules reads `AGENTS.md` and works in its own virtual machine, using the scripts and tests directly. Agent pull requests say that they are agent-authored and name the tool; a human maintainer merges.

12.4 A worked request

Worked example 12.1 (From question to merged result). A student asks: “What is the zero-sound velocity at 15 bar with F_0^s and F_1^s , at Padé order 6, and how far is it from the $[4/4]$ value?” A good issue states the inputs with their source (`he3_landau_parameters` at 15 bar, from [7]), the quantity (the root of Eq. (3.8) at $M = 6$ and $M = 4$, and their relative difference), and the evidence level (Level 1 for the roots, Level 2 for a 60-digit comparison). An agent then adds a test or script, commits a small JSON result under `alexandrie_data/`, and opens a pull request whose description quotes the numbers and their level. Continuous integration re-runs everything.

Exercises

Exercise 12.1. Rewrite this request so that an agent can execute it: “Can you check whether zero sound works for helium at high pressure?”

Appendix A

Solutions to the exercises

1.1. `float(1/1001)` is close to 0.000999001, and the best rational approximation with denominator at most 1000 is $1/1000$. The parameter has been silently replaced by a different one, with a relative change of 10^{-3} . Worse than the size of the change is that nothing reports it: every later exact computation is exact for the wrong input.

1.2. (a) Level 1. (b) Level 2. (c) Level 3 (Theorem 3.1). (d) Level 4: it is a statement about measurement; the project's model reproduces it only with F_1^s . (e) No evidence: it was retracted (R3).

1.3. A table test checks that code and table agree. If both came from the same wrong source, or the table was invented and the code made to match it, the test passes. A closed-form test compares against an independent derivation, so a wrong table fails.

2.1. With $a_0 = 1, a_1 = 1, a_2 = 1/2$: $a_2 + q_1 a_1 = 0$ gives $q_1 = -1/2$, then $p_0 = 1, p_1 = a_1 + q_1 a_0 = 1/2$. So $e^x \approx (1 + x/2)/(1 - x/2)$, which is 3 at $x = 1$, against $e = 2.71828$ and the Taylor value 2.5. Both use three coefficients; here the Taylor polynomial is slightly closer, but the Padé form keeps the symmetry $R(-x) = 1/R(x)$ of the exponential.

2.2. The conditions for $n = 3, 4$ are $a_3 + q_1 a_2 + q_2 a_1 = 0$ and $a_4 + q_1 a_3 + q_2 a_2 = 0$, i.e. $\frac{1}{7} + \frac{q_1}{5} + \frac{q_2}{3} = 0$ and $\frac{1}{9} + \frac{q_1}{7} + \frac{q_2}{5} = 0$. Their solution is $q_1 = -10/9, q_2 = 5/21$. Then $p_0 = 0, p_1 = a_1 = 1/3, p_2 = a_2 + q_1 a_1 = \frac{1}{5} - \frac{10}{27} = -\frac{23}{135}$.

2.3. For $A = 1 + u^2$ (so $a_1 = 0, a_2 = 1$), the $[1/1]$ condition $a_2 + q_1 a_1 = 0$ reads $1 = 0$: no solution. An exact engine must report that the approximant does not exist. A least-squares fallback would return *some* rational function that satisfies no order condition, and every root computed from it would look exact while meaning nothing.

3.1. $\frac{s}{2} \ln \frac{s+1}{s-1} = s \operatorname{artanh}(1/s) = \sum_{k \geq 0} \frac{s^{-2k}}{2k+1} = 1 + \sum_{k \geq 1} \frac{u^k}{2k+1}$. Subtracting 1 gives $a_k = 1/(2k+1)$: $1/3, 1/5, 1/7$.

3.2. $Q - FP = 1 - \frac{3u}{5} - \frac{Fu}{3} = 0$ gives $u = \frac{1}{3/5 + F/3} = \frac{15}{9 + 5F}$. Then $u < 1$ iff $15 < 9 + 5F$ iff $F > 6/5$; and $u > 0$ for $F > 0$.

3.3. Lower edge: $2e^{-(2+2/9.3)} = 0.2183$. Upper edge: 9.3. The $M = 1$ value gives $s-1 = 0.9235$, inside. The upper edge comes from $\ln y \leq y - 1$, which is tight only for y near 1, i.e. for s large; at strong coupling the root is at moderate s , where the inequality is far from equality.

3.4. With $F_1^s = 0$, $a = 1$ and the polynomial is $P F_0^s u - Q u = u(F_0^s P - Q)$. The extra root is $u = 0$, i.e. $s = \infty$, which is excluded because admissible roots must lie in the open interval $(0, 1)$.

4.1. The bracket is $\frac{5K-3}{2K} + \frac{1-K}{2K^2}|v|^2$. For $K \leq 1$ the second coefficient is nonnegative, so the minimum is at $v = 0$, where the value is $(5K - 3)/(2K)$. It is nonnegative iff $K \geq 3/5$.

4.2. $\nabla M = -vM$, so $|\nabla M|^2/M = |v|^2 M$ and $I(M) = \int |v|^2 M = d$. For $d = 3$, $I = 3$.

4.3. $1 - K = 0.35 e^{-t/6} < 10^{-3}$ iff $t > 6 \ln 350 \approx 35.1$.

5.1. $\gamma = 1 - 2\nu \in (-3, -2\sqrt{2}]$ iff $2\nu - 1 \in [2\sqrt{2}, 3]$ iff $\nu \in [(1 + 2\sqrt{2})/2, 2)$, and $(1 + 2\sqrt{2})/2 = 1.9142$. The criterion $|\gamma| \leq 2\sqrt{4m/M}$ reads $2\nu - 1 \leq 4\sqrt{m/M}$, i.e. $m/M \geq (2\nu - 1)^2/16$; at $\nu \rightarrow 2$ this is $9/16$.

5.2. $2\sqrt{3 \times 0.99654} = 3.458$; the ceiling is $2\sqrt{3} = 3.464$.

5.3. Write $e^{-\alpha t} - e^{-\beta t} = \int_{\alpha}^{\beta} t e^{-\lambda t} d\lambda$. By Tonelli, the integral equals $\int_{\alpha}^{\beta} \int_0^{\infty} t^{-s} e^{-\lambda t} dt d\lambda = \Gamma(1-s) \int_{\alpha}^{\beta} \lambda^{s-1} d\lambda = \frac{\Gamma(1-s)}{s} (\beta^s - \alpha^s)$, and $\Gamma(1-s)/s = -\Gamma(-s)$, giving $\Gamma(-s)(\alpha^s - \beta^s)$. As $s \rightarrow 1$ the integrand behaves like $(\beta - \alpha)t^{-s}$ near $t = 0$, close to the non-integrable t^{-1} , so adaptive quadrature misses mass near the origin; this is the underestimate of Λ_b found in the re-examination.

5.4. $m = \inf_{\theta} \beta/\beta_0$ over all angles, and the infimum over a subset is at least the infimum over the whole set; likewise the sampled supremum is at most M . So the sampled m/M is at least the true one, and so is the bound.

6.1. $\text{Si}^2 = t^2 - \frac{2}{18}t^4 + \left(\frac{1}{324} + \frac{2}{600}\right)t^6 + \dots$, and $\frac{1}{324} + \frac{1}{300} = \frac{13}{2025}$. Halving: $\frac{1}{2}t^2 - \frac{1}{18}t^4 + \frac{13}{4050}t^6$, as in Eq. (6.1).

6.2. Complete the square: $\int e^{-ikvt} (2\pi)^{-1/2} e^{-v^2/2} dv = e^{-k^2 t^2/2}$, the characteristic function of the standard normal distribution.

6.3. The area of the zone $z \in [a, b]$ on the unit sphere is $2\pi(b - a)$, proportional to its height (Archimedes). Dividing by 4π gives the uniform density $1/2$ on $[-1, 1]$.

7.1. For example:

```
def test_m1_closed_form_and_float_refusal():
    a = LinearResponseStage().landau_zero_sound_kernel(order=4)
    k = KineticStage()
    r = k.solve_zero_sound_root(a, sp.Rational(7, 2), M=1)
    F = sp.Rational(7, 2)
    assert sp.simplify(r["u_exact"]) == 15 / (9 + 5 * F)
    with pytest.raises(ScientificHonestyException):
        k.solve_zero_sound_root(a, 3.5, M=1)
```

Here $u = 30/53$.

7.2. It fails the naive expectation that a solver should find a root, so it looks like a bug report. Deleting it would allow a later change to return a spurious root at weak coupling, silently re-introducing the retracted capability.

8.1. A root needs $g_3(s) = 1/F$. For $s > 1$, $g_3(s) > 0$ (`g3_pos`), so $F < 0$ is impossible. For $F = 0$, Lean defines $1/0 = 0$, and $g_3(s) = 0$ is also impossible. So every case that satisfies the hypotheses already has $F > 0$.

8.2. Nothing beyond its hypothesis: after unfolding, hypothesis and conclusion are the same statement. It compiles, and it is “verified”, and it carries no information.

9.1. $m^*/m = (2/\pi^2) \times 2.7411 \times 4.9569 = 2.7534$, and $F_1^s = 3(2.7534 - 1) = 5.260$.

9.2. $c_1 = 60.042\sqrt{11.2788 \times 2.7534/3} = 60.042 \times 3.2174 = 193.2$ m/s, as in Table 9.1.

9.3. $\lambda\varepsilon_{\maxon} - 2\lambda\Delta = \lambda(\varepsilon_{\maxon} - 2\Delta)$, and $\lambda > 0$ preserves the sign.

9.4. $0.0106/0.0005 \approx 21$. It assumes the quoted uncertainty is a Gaussian standard deviation that captures all errors; a systematic energy offset that is not proportional would not be covered by the recalibration lemma either.

10.1. $m_4 - 15 = -15(1 - K)^2 = -15(1 - K_0)^2 e^{-t/3}$, whose derivative is $-\frac{1}{3}(m_4 - 15)$; with $m_2 = 3$, $\frac{5}{3}m_2^2 = 15$, so the moment equation holds.

10.2. $\text{Var} = 945 - 225 = 720$, so the standard error is $\sqrt{720/10^6} \approx 0.027$, relative to 15 about 0.18%.

10.3. $\ln(1 - \Delta t/3) = -\Delta t/3 - \Delta t^2/18 - \dots$, so the ratio of the two is about $e^{-T\Delta t/18} \approx e^{-0.017}$ on the factor e^{-10} : a relative change of 1.7% in the *deviation* $m_4 - 15$, which is itself tiny by then. The script reports the largest such bias over the whole run: 1.0×10^{-4} relative on m_4 . It is exact because the per-step contraction of the moment is known in closed form for this algorithm.

11.1. A JSON number is parsed as a binary float before the server sees it, so the exact value intended by the user is already lost. The string “9.3” is converted exactly to 93/10.

12.1. For example: “Compute the undamped zero-sound root s of Eq. (3.8) at $M = 4$ for the tabulated pressures 20, 25 and 29 bar (`he3_landau_parameters`), and the ratio c_0/c_1 . Evidence level: Level 1 for the roots, Level 2 comparison with a 60-digit solution. Deliverable: a script under `verification/`, a JSON summary under `alexandrie_data/`, and a test.”

Appendix B

Glossary

Algebraic number

A root of a nonzero polynomial with rational coefficients.

BKW solution

The Bobylev–Krook–Wu exact solution of the homogeneous Boltzmann equation for Maxwell molecules.

DSMC

Direct Simulation Monte Carlo: particles move and collide at random with the rates of the kinetic equation.

Evidence level

The kind of check behind a claim: 1 exact computation, 2 independent validation, 3 Lean proof, 4 comparison with data.

Fisher information

$I(f) = \int |\nabla f|^2 / f$; a measure of the roughness of f .

First sound

Ordinary, collision-dominated sound.

Landau parameters

$F_l^{s,a}$: Legendre components of the quasiparticle interaction.

Maxwell molecules

Collision kernels independent of the relative speed ($\gamma = 0$).

MCP

Model Context Protocol, a standard way for AI assistants to call local tools.

Padé approximant

A rational function whose Taylor series matches a given series to a given order.

Phase mixing

Decay of a density perturbation by filamentation in velocity, without dissipation.

Retraction

A claim withdrawn by the project and recorded in `RETRACTIONS.md`.

Roton, maxon

The local minimum and maximum of the ^4He excitation spectrum.

sorryAx

The axiom that Lean’s `sorry` introduces; its presence means a proof is incomplete.

Zero sound

A collisionless collective mode of a Fermi liquid.

Bibliography

- [1] C. Villani, *Fisher Information in Kinetic Theory*, arXiv:2501.00925 (2025). Lecture notes; Section 22, Theorem 22.6 and Remarks 22.7–22.8.
- [2] C. Imbert, L. Silvestre and C. Villani, *On the monotonicity of the Fisher information for the Boltzmann equation*, arXiv:2409.01183 (2024).
- [3] L. Silvestre, *Collision kernels* (Julia code), <https://github.com/luissilvestre/collisionkernel>, commit 01a9d44 (2024).
- [4] H. Godfrin, K. Beauvois, A. Sultan, E. Krotscheck, J. Dawidowski, B. Fåk and J. Ollivier, *Dispersion relation of Landau elementary excitations and thermodynamic properties of superfluid ^4He* , Phys. Rev. B **103**, 104516 (2021), arXiv:2012.09067, with ancillary data files.
- [5] K. Beauvois, J. Dawidowski, B. Fåk, H. Godfrin, E. Krotscheck, J. Ollivier and A. Sultan, *Microscopic dynamics of superfluid ^4He : a comprehensive study by inelastic neutron scattering*, Phys. Rev. B **97**, 184520 (2018).
- [6] H. Godfrin and E. Krotscheck, *The Dynamics of Quantum Fluids*, arXiv:2206.06039 (2022).
- [7] M. Kollar and D. Vollhardt, *Thermodynamically consistent equilibrium properties of normal-liquid Helium-3*, Phys. Rev. B **61**, 15347 (2000), arXiv:cond-mat/9906222.
- [8] D. Greywall, Phys. Rev. B **27**, 2747 (1983). The specific-heat data from which the tables of [7] are derived.
- [9] M. Krook and T. T. Wu, *Formation of Maxwellian tails*, Phys. Rev. Lett. **36**, 1107 (1976).
- [10] D. Pines and P. Nozières, *The Theory of Quantum Liquids*, Vol. I, W. A. Benjamin, New York (1966).
- [11] G. A. Baker Jr. and P. Graves-Morris, *Padé Approximants*, 2nd ed., Cambridge University Press (1996).
- [12] The mathlib Community, *The Lean Mathematical Library*, CPP 2020.